

BEP-007: Tuplas e Dicionários

Conheça outras estruturas de dados fundamentais em Python

Acesso a Dicionários

Métodos de Acesso

```
pessoa = {
    "nome": "Ana Silva",
    "idade": 28,
    "profissao": "Designer",
    "cidade": "Salvador"
}

# 1. Acesso direto com []
nome = pessoa["nome"]
idade = pessoa["idade"]
print(f"Nome: {nome}, Idade: {idade}")

# 2. Método get() - mais seguro
nome = pessoa.get("nome")
telefone = pessoa.get("telefone") # Retorna None se não existir
telefone_default = pessoa.get("telefone", "Não informado")

print(f"Nome: {nome}")
print(f"Telefone: {telefone}") # None
print(f"Telefone: {telefone_default}") # Não informado
```



Acesso Seguro

```
# Comparação entre acesso direto e get()
dados = {"nome": "João", "idade": 25}

# Acesso direto - pode gerar erro
# profissao = dados["profissao"] # KeyError!

# Acesso seguro com get()
profissao = dados.get("profissao", "Não informado")
print(f"Profissão: {profissao}") # Não informado

# Verificando existência antes de acessar
if "profissao" in dados:
    profissao = dados["profissao"]
```



```
else:
    profissao = "Não informado"
print(f"Profissão: {profissao}")
```

Iterando sobre Dicionários

```
contato = {
    "nome": "Maria",
    "telefone": "99999-8888",
    "email": "maria@email.com",
    "cidade": "Salvador"
}

# Iterando sobre chaves
print("Chaves:")
for chave in contato.keys():
    print(f" {chave}")

# Iterando sobre valores
print("\nValores:")
for valor in contato.values():
    print(f" {valor}")

# Iterando sobre pares chave-valor
print("\nPares chave-valor:")
for chave, valor in contato.items():
    print(f" {chave}: {valor}")

# Iterando diretamente (sobre chaves)
print("\nIteração direta:")
for chave in contato:
    print(f" {chave}: {contato[chave]}")
```



Acesso a Dados Aninhados

```
# Dicionário aninhado
empresa = {
    "nome": "TechCorp",
    "funcionarios": {
        "joao": {
            "cargo": "Desenvolvedor",
            "salario": 5000,
            "departamento": "TI"
        },
        "maria": {
            "cargo": "Designer",
            "salario": 4000,
            "departamento": "Design"
        }
    }
}
```



```
# Acesso a dados aninhados
print(f"Empresa: {empresa['nome']}")
print(f"João - Cargo: {empresa['funcionarios']['joao']['cargo']}")
print(f"João - Salário: R$ {empresa['funcionarios']['joao']['salario']}")

# Acesso seguro a dados aninhados
joao_salario = empresa.get("funcionarios", {}).get("joao", {}).get("salario", 0)
print(f"Salário do João: R$ {joao_salario}")

# Iterando sobre funcionários
print("\nFuncionários:")
for nome, dados in empresa["funcionarios"].items():
    print(f" {nome.title():} {dados['cargo']} - R$ {dados['salario']}")
```

Verificações e Testes

```
dados = {"a": 1, "b": 2, "c": 3}

# Verificando existência de chaves
print(f"'a' existe: {'a' in dados}") # True
print(f"'d' existe: {'d' in dados}") # False

# Verificando existência de valores
print(f"Valor 2 existe: {2 in dados.values()}") # True
print(f"Valor 4 existe: {4 in dados.values()}") # False

# Verificando existência de itens
print(f"('a', 1) existe: {'a', 1} in dados.items()") # True

# Usando not in
print(f"'d' não existe: {'d' not in dados}") # True
```

Exemplo: Sistema de Login

```
# Sistema de usuários
usuarios = {
    "admin": {
        "senha": "123456",
        "nivel": "administrador",
        "ativo": True
    },
    "usuario1": {
        "senha": "senha123",
        "nivel": "usuario",
        "ativo": True
    }
}

def verificar_login(usuario, senha):
    # Verificando se usuário existe
    if usuario not in usuarios:
```

```
    return False, "Usuário não encontrado"

# Acessando dados do usuário
dados = usuarios[usuario]

# Verificando senha e status
if dados["senha"] == senha and dados["ativo"]:
    return True, dados["nivel"]
elif not dados["ativo"]:
    return False, "Usuário inativo"
else:
    return False, "Senha incorreta"

# Testando o sistema
resultado, info = verificar_login("admin", "123456")
print(f"Login admin: {'Sucesso' if resultado else 'Falha'} - {info}")
```

Dicas de Acesso

- **Use get():** Para acesso seguro com valor padrão
- **Verifique existência:** Use `in` antes de acessar
- **Iteração eficiente:** Use `items()` para pares chave-valor
- **Dados aninhados:** Acesse nível por nível

